

# Java Lab Record: Multi-Threading - Seat Reservation System

## Objective / Aim

To design and implement a seat **reservation system** in Java using **multiple threads**, where several person threads try to book seats and **synchronization** ensures that only one person thread can use the `reserve` method at any time.

## Problem Statement

Implement a reservation system which allows persons to book seats. Define a `reserve` method with some initial number of seats. Now create one or more person threads to book seats. At any time it should allow **only one person thread** to access the `reserve` method.

### Expected Output

```
Person-1 entered.
Available seats: 10 Requested seats: 5
Seat Available. Reserve now :-)
5 seats reserved.
Person-1 leaving.
-----
Person-2 entered.
Available seats: 5 Requested seats: 2
Seat Available. Reserve now :-)
2 seats reserved.
Person-2 leaving.
-----
Person-3 entered.
Available seats: 3 Requested seats: 4
Requested seats not available :-)
Person-3 leaving.
```

**Note:** The statement says "initially 100 seats", but the expected output starts with **10** available seats. The program uses **10** so that its output matches the expected output. Change `availableSeats = 10` to `100` if you want 100 seats.

---

## System Requirements

### Hardware

Standard PC / Laptop

Intel Core i3 / i5 processor or equivalent  
Minimum 4 GB RAM  
Minimum 500 MB free storage

## Software

Operating System : Windows / Linux / macOS  
Java Development Kit : JDK 17 or JDK 21  
Text Editor or IDE : Eclipse, IntelliJ IDEA, NetBeans, or VS Code  
Command Prompt / Terminal

---

## ❏ Algorithm / Logic Description

1. Create a class named ReservationSystem.
2. Declare a shared data member availableSeats (initially 10).
3. Create a method reserve(person, requestedSeats) and mark it 'synchronized' so that only ONE thread can run it at a time.
4. Inside reserve():
  - a. Print "<person> entered."
  - b. Print the available seats and the requested seats.
  - c. If requestedSeats <= availableSeats:  
    print "Seat Available. Reserve now :-)",  
    subtract the seats, print "<n> seats reserved."  
    Else:  
        print "Requested seats not available :-)"
  - d. Print "<person> leaving."
5. In main(), create ONE ReservationSystem object that all threads share.
6. Create three person threads (Person-1, Person-2, Person-3) with the seats each one wants (5, 2 and 4).
7. Start each thread with start() and wait for it with join().
8. Print a separator line between the persons.

## 🌐 Real-time Application

Railway, bus and flight ticket booking, cinema seat booking, online shopping stock updates, and bank account transactions, anywhere many users change the same data at the same time.

---

---

# ReservationSystem.java

## 📄 Source Code

```
package beyond;  
  
public class ReservationSystem {
```

```

    // This value is shared by all person threads, so bookings must update it
safely.
    private int availableSeats = 10;

    // synchronized locks this ReservationSystem object for the whole check-
and-book operation.
    // Without the lock, two threads could both see the same seats as
available and overbook.
    private synchronized void reserve(String person, int requestedSeats) {
        System.out.println(person + " entered.");
        System.out.println("Available seats: " + availableSeats
            + " Requested seats: " + requestedSeats);

        if (requestedSeats <= availableSeats) {
            System.out.println("Seat Available. Reserve now :-)");
            availableSeats -= requestedSeats;
            System.out.println(requestedSeats + " seats reserved.");
        } else {
            System.out.println("Requested seats not available :-)");
        }

        System.out.println(person + " leaving.");
    }

    public static void main(String[] args) throws InterruptedException {
        ReservationSystem reservation = new ReservationSystem();

        // Each lambda is the task (Runnable) that a separate thread will
execute.
        Thread person1 = new Thread(() -> reservation.reserve("Person-1",
5));
        Thread person2 = new Thread(() -> reservation.reserve("Person-2",
2));
        Thread person3 = new Thread(() -> reservation.reserve("Person-3",
4));

        // start() runs the task on a new thread; join() makes main wait for
that thread to finish.
        // Joining in this order keeps the sample output predictable;
synchronized still protects
        // the shared seat count if the threads are allowed to run at the
same time.
        person1.start();
        person1.join();
        System.out.println("-");

        person2.start();
        person2.join();
        System.out.println("-");

        person3.start();
        person3.join();
    }
}

```

## 1. Concept and Exception in Simple Words

## Concepts used

|                 |                                                                                                                               |
|-----------------|-------------------------------------------------------------------------------------------------------------------------------|
| Thread          | -> a separate path of execution. Each person is one thread, so several persons can try to book at the same time.              |
| Runnable/lambda | -> the TASK a thread runs. The lambda<br>() -> reservation.reserve("Person-1", 5)<br>is the task given to each Thread object. |
| start()         | -> creates a NEW thread and runs the task on it.<br>(Calling run() directly would NOT create a thread.)                       |
| join()          | -> makes the calling thread (main) WAIT until that thread has finished.                                                       |
| synchronized    | -> a lock. Only ONE thread at a time can run a synchronized method of the same object; the others wait.                       |
| Shared data     | -> availableSeats is one variable used by all threads.                                                                        |

## Why synchronization is needed (race condition)

Booking has two steps: **check** the seats, then **subtract** them. Without a lock, two threads could both check at the same time and both see "enough seats", so the seats get **overbooked**. This kind of bug is called a **race condition**. With `synchronized`, the whole check-and-book steps run as **one unit**: a second person must wait outside until the first has left.

|                                         |                              |
|-----------------------------------------|------------------------------|
| Without synchronized (possible problem) | With synchronized (safe)     |
| Seats = 5                               | Seats = 5                    |
| Person-A checks: 5 >= 4 OK              | Person-A takes the lock      |
| Person-B checks: 5 >= 4 OK              | Person-A books 4, seats = 1  |
| Person-A books 4 -> seats = 1           | Person-A releases the lock   |
| Person-B books 4 -> seats = -3 (WRONG)  | Person-B takes the lock      |
|                                         | Person-B checks: 1 >= 4 ? NO |

The lock belongs to the **object** (`this`). All three threads use the **same** reservation object, so they all compete for the same lock. If every thread had its own `ReservationSystem` object, the lock would not stop them.

## What the program really does with `join()`

The program calls `start()` and then `join()` for each person **one after another**. So each person finishes before the next one starts. That keeps the output in the same order as the expected output. In this form the threads never actually overlap, so the lock is not tested. It is still correct and still required, because it protects the seats whenever threads do run together.

**Tested by running:** when all three threads were started first and joined afterwards, the order changed between runs (in some runs Person-3 entered before Person-2). But every person's block of messages stayed together, and the seat count never went wrong. That is the work of `synchronized`.

```
// Optional version: start all threads first, then wait for all of them
person1.start();
person2.start();
person3.start();
```

```
person1.join();
person2.join();
person3.join();
```

## Thread priorities

Every thread has a priority from 1 to 10 (`Thread.MIN_PRIORITY = 1`, `Thread.NORM_PRIORITY = 5`, `Thread.MAX_PRIORITY = 10`). A higher priority is only a **hint** to the scheduler, not a guarantee. **This program does not set any priority**, so all three threads use the default, 5. To use priorities:

```
person1.setPriority(Thread.MAX_PRIORITY);    // 10
person3.setPriority(Thread.MIN_PRIORITY);    // 1
```

## Exception in simple words

No exception is thrown or caught inside the booking logic. Two exceptions are related to this program:

| Exception                                | Kind      | Where it comes from                                                                                                                                                     |
|------------------------------------------|-----------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>InterruptedException</code>        | Checked   | <code>join()</code> can throw it if the waiting thread is interrupted. <code>main()</code> declares throws <code>InterruptedException</code> instead of using try-catch |
| <code>IllegalThreadStateException</code> | Unchecked | Raised if <code>start()</code> is called twice on the same Thread object. This program starts each thread once, so it never happens                                     |

A race condition (the bug explained above) is **not** an exception. Java does not warn you about it. The program simply gives wrong results, which is why the lock is needed.

## 2. Step-wise Execution

- `main()` starts. `availableSeats` is 10 (the `ReservationSystem` object 'reservation' is created once and shared).
- Three Thread objects are created. Each holds a task (lambda) that calls `reservation.reserve(name, seats)`. Nothing runs yet.
- `person1.start()` -> a new thread runs `reserve("Person-1", 5)`:
  - the thread takes the lock on 'reservation'
  - prints "Person-1 entered."
  - prints "Available seats: 10 Requested seats: 5"
  - `5 <= 10` -> prints "Seat Available. Reserve now :-)"
  - `availableSeats = 10 - 5 = 5`
  - prints "5 seats reserved."
  - prints "Person-1 leaving." and releases the lock
- `person1.join()` -> main waits until Person-1's thread has finished.
- main prints the separator "-".
- `person2.start()` -> runs `reserve("Person-2", 2)`:

```

        - prints "Available seats: 5 Requested seats: 2"
        - 2 <= 5 -> seats reserved, availableSeats = 5 - 2 = 3
        - prints "2 seats reserved." and "Person-2 leaving."
7. person2.join(), then main prints "-".
8. person3.start() -> runs reserve("Person-3", 4):
    - prints "Available seats: 3 Requested seats: 4"
    - 4 <= 3 ? NO -> prints "Requested seats not available :-)"
    - availableSeats stays 3
    - prints "Person-3 leaving."
9. person3.join() -> main waits, then the program ends.

```

## Seat count after each person

```

Start      : 10 seats
Person-1   : asks 5 -> available (10 >= 5) -> 5 seats left
Person-2   : asks 2 -> available (5 >= 2) -> 3 seats left
Person-3   : asks 4 -> NOT available (3 < 4) -> 3 seats left

```

## 3. Algorithm

```

START
1. Create ReservationSystem object 'reservation' (availableSeats = 10)
2. Create thread person1 to run reserve("Person-1", 5)
3. Create thread person2 to run reserve("Person-2", 2)
4. Create thread person3 to run reserve("Person-3", 4)
5. START person1; WAIT for person1 to finish (join); PRINT "-"
6. START person2; WAIT for person2 to finish (join); PRINT "-"
7. START person3; WAIT for person3 to finish (join)
STOP

reserve(person, requestedSeats) [synchronized: one thread at a time]
1. PRINT person + " entered."
2. PRINT "Available seats: " + availableSeats + " Requested seats: " +
requestedSeats
3. IF requestedSeats <= availableSeats:
    PRINT "Seat Available. Reserve now :-)"
    availableSeats = availableSeats - requestedSeats
    PRINT requestedSeats + " seats reserved."
    ELSE:
    PRINT "Requested seats not available :-)"
4. PRINT person + " leaving."

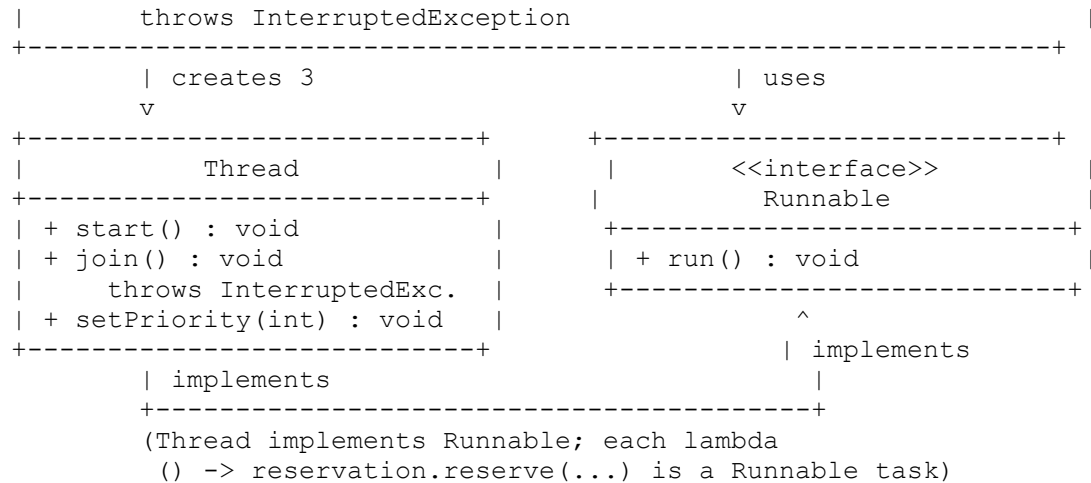
```

## 4. UML Class Diagram

```

+-----+
|                                     |
|               ReservationSystem   |
|               (package: beyond)   |
+-----+
| - availableSeats : int = 10        | (shared by all threads)
+-----+
| - reserve(person : String,         | requestedSeats : int) : void
|     {synchronized}                |
| + main(args : String[]) : void {static}
+-----+

```



Three Thread objects share ONE ReservationSystem object:

```

person1 (Thread) ----+
person2 (Thread) ----+----> reservation : ReservationSystem (one lock)
person3 (Thread) ----+

```

**Legend:** - private, + public, {static} belongs to the class itself.

## Compilation and Execution

Folder structure:

```

project/
|-- beyond/
    |-- ReservationSystem.java

```

Compile:

```
javac -d out beyond/ReservationSystem.java
```

Run:

```
java -cp out beyond.ReservationSystem
```

## 📄 Sample Output

This is the actual output of the program:

```

Person-1 entered.
Available seats: 10 Requested seats: 5
Seat Available. Reserve now :-)
5 seats reserved.
Person-1 leaving.
-
Person-2 entered.

```

```
Available seats: 5 Requested seats: 2
Seat Available. Reserve now :-)
2 seats reserved.
Person-2 leaving.
-
Person-3 entered.
Available seats: 3 Requested seats: 4
Requested seats not available :-)
Person-3 leaving.
```

## Output:

```
PS C:\Users\Admin\OneDrive\Documents\basicjava> c::; cd 'c:\Users\Admin\OneDrive\Documents\basicjava'; & 'C:\Program Files\Java\jdk-26.0.1\bin\
java.exe' '-enable-preview' '-XX:+ShowCodeDetailsInExceptionMessages' '-cp' 'C:\Users\Admin\OneDrive\Documents\basicjava\bin' 'beyond.Reservat
ionSystem'
Person-1 entered.
Available seats: 10 Requested seats: 5
Seat Available. Reserve now :-)
5 seats reserved.
Person-1 leaving.
-
Person-2 entered.
Available seats: 5 Requested seats: 2
Seat Available. Reserve now :-)
2 seats reserved.
Person-2 leaving.
-
Person-3 entered.
Available seats: 3 Requested seats: 4
Requested seats not available :-)
Person-3 leaving.
```

The separator printed by the program is a single -. The expected output in the question shows a long line of dashes. To match it exactly, change `System.out.println("-");` to `System.out.println("-----");`.

---

## ✦ Result

The Seat Reservation System was successfully implemented in Java using multithreading. Three person threads were created to book seats from a shared `ReservationSystem` object, and the synchronized `reserve` method ensured that only one person thread could check and book seats at any time, so the seat count stayed correct. The program booked 5 seats for Person-1, 2 seats for Person-2, and correctly reported that Person-3's request for 4 seats could not be met, matching the expected output.